

ORDINO

Git & GitHub Workflow Guide

Required commands only - explained through the real Pawan and Himani workflow

PAWAN KSHETRI

Frontend: web, Android, and iOS

HIMANI SINGWAL

Backend APIs and database

ORDINO TEAM WORKFLOW



Pawan Kshetri — FRONTEND



Himani Singwal — BACKEND + DATABASE



DEVELOP

Daily work



STAGING

Testing + Integration



MAIN

Stable Production

DAILY COMMANDS

```
1 | git switch develop
2 | git pull --rebase origin develop
3 | git add <files>
4 | git commit -m "feat(scope): message"
5 | git pull --rebase origin develop
6 | git push origin develop
```

RULES

- ✓ Work only on develop
- ✓ Pull before coding
- ✓ No direct push to staging or main
- ✓ develop → staging: Pull Request
- ✓ staging → main: Pull Request

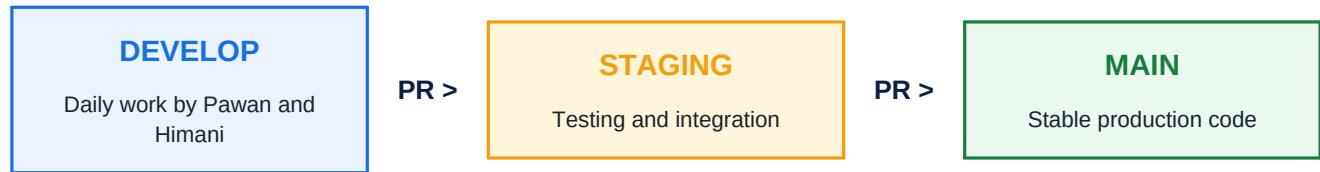
ONE-LINE RULE

Daily code goes to **develop**. Tested code moves to **staging** through a pull request. Approved stable code moves to **main** through another pull request.

Repository: <https://github.com/Pawankshetri11/Ordino>

Prepared for the two-developer Ordino team | 25 August 2026

1. The only branch model you need



COMMAND	KYA KARTA HAI
develop	Pawan aur Himani dono ka daily working branch.
staging	Complete milestone ko combined test karne ka branch. Direct coding nahi.
main	Stable production-ready code. Direct coding ya direct push nahi.

IMPORTANT

Exactly three branches rakho. Feature branches nahi. Normal coding sirf **develop** par hogi; **staging** aur **main** protected promotion branches hain.

One-time: staging branch kaise create hota hai

```
git switch develop
git pull --ff-only origin develop
git switch -c staging
git push -u origin staging
git switch develop
```

COMMAND	KYA KARTA HAI
git switch -c staging	Local staging branch create karta hai aur usi par switch karta hai.
git push -u origin staging	Staging ko GitHub par publish karta hai aur tracking set karta hai.
git switch develop	One-time setup ke baad daily work ke liye develop par wapas lata hai.

DO NOT REPEAT

Staging branch already GitHub par create ho chuki hai. Ye creation commands dobara chalane ki zarurat nahi.

2. Real daily scenario: Pawan push, Himani pull and push

Step A - Pawan starts frontend work

```
git switch develop
git pull --rebase origin develop
git status
# Pawan edits frontend files
git add frontend
git commit -m "feat(frontend): add login screen"
git pull --rebase origin develop
git push origin develop
```

Result: Pawan ka frontend commit GitHub ke **develop** branch par pahunch gaya.

Step B - Himani gets Pawan's work before backend work

```
git switch develop
git pull --rebase origin develop
# Himani now has Pawan's frontend commit
# Himani edits backend/database files
git add backend
git commit -m "feat(backend): add login endpoint"
git pull --rebase origin develop
git push origin develop
```

Result: GitHub ke **develop** branch mein ab Pawan ka frontend aur Himani ka backend dono hain.

Step C - Pawan gets Himani's latest backend work

```
git switch develop
git pull --rebase origin develop
```

THE HABIT THAT PREVENTS MOST PROBLEMS

Coding start karne se pehle pull karo. Commit ke baad push se pehle dobara pull karo.

3. Required Git commands and what they do

COMMAND	KYA KARTA HAI
<code>git status</code>	Current branch aur changed/untracked files dikhata hai.
<code>git branch -vv</code>	Local branches aur unke remote tracking branches dikhata hai.
<code>git switch develop</code>	Daily working branch develop par le jata hai.
<code>git pull --rebase origin develop</code>	GitHub ke latest develop commits local commits ke neeche clean order mein lata hai.
<code>git diff</code>	Unstaged code changes review karne ke liye.
<code>git add <specific-files></code>	Sirf selected files ko next commit ke liye stage karta hai.
<code>git diff --staged</code>	Commit se pehle staged changes ka final review.
<code>git commit -m "message"</code>	Staged changes ka local checkpoint/history entry banata hai.
<code>git push origin develop</code>	Local develop commits GitHub develop branch par bhejta hai.
<code>git log --oneline -10</code>	Recent 10 commits short form mein dikhata hai.
<code>git fetch origin</code>	Remote information download karta hai, working files change nahi karta.
<code>git rebase --continue</code>	Conflict resolve aur stage hone ke baad paused rebase continue karta hai.
<code>git rebase --abort</code>	Confusing rebase ko safely cancel karke previous state mein lautata hai.

Required GitHub / PR commands

COMMAND	KYA KARTA HAI
<code>gh auth login</code>	GitHub CLI ko account se login karta hai. Usually one-time setup.
<code>gh pr create --base staging --head develop</code>	Develop se staging promotion pull request banata hai.
<code>gh pr create --base main --head staging</code>	Staging se main production pull request banata hai.

GIT VS GITHUB

Commit local Git history banata hai. Push us history ko GitHub par bhejta hai. Pull GitHub ke latest commits local machine par lata hai. Pull request ek branch ko review ke baad doosri branch mein merge karne ka GitHub process hai.

4. Develop se staging kaise le jana hai

Auth milestone complete hone ka example:

- Pawan ne frontend auth code develop par push kar diya.
- Himani ne backend aur database auth code develop par push kar diya.
- Dono ne latest develop pull karke combined behavior check kiya.
- Ab develop ka complete snapshot staging testing ke liye promote hoga.

Create the promotion pull request

```
git switch develop
git pull --rebase origin develop
git status
gh pr create --base staging --head develop \
--title "release: promote develop to staging"
```

PR DIRECTION

Base branch **staging** hai. Compare/head branch **develop** hai. Meaning: develop ka code staging mein jana hai.

Review and merge

- Other developer combined frontend/backend diff review kare.
- Required checks pass hone chahiye.
- GitHub par Create a merge commit use karo.
- Staging par test karo; staging par direct code edit ya push mat karo.

Staging par bug mile to

```
git switch develop
git pull --rebase origin develop
# Fix the bug on develop
git add <specific-files>
git commit -m "fix(scope): fix staging issue"
git pull --rebase origin develop
git push origin develop
# Then create develop -> staging PR again
```

5. Staging se main production release

Main tabhi update hoga jab staging testing complete ho aur Pawan + Himani dono approve karein.

Create the production pull request

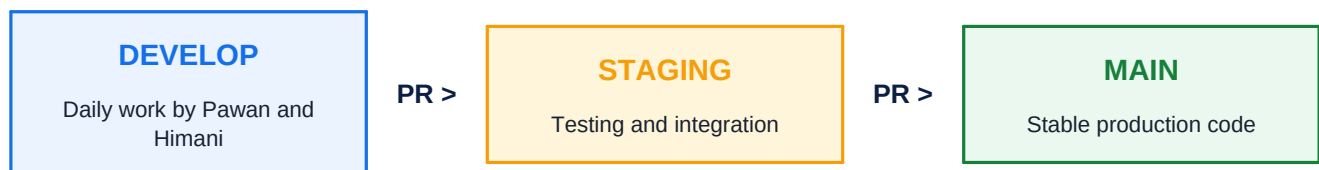
```
gh pr create --base main --head staging \
--title "release: promote staging to main"
```

PR DIRECTION

Base branch **main** hai. Compare/head branch **staging** hai. Meaning: tested staging code production main mein jana hai.

Production release checklist

- Frontend type checks/builds passed.
- Backend build and relevant tests passed.
- Database migrations reviewed and tested, when database is introduced.
- Frontend and backend API contract matches.
- No .env file, password, token, or private credential committed.
- Both developers reviewed and approved the PR.



NEVER SKIP STAGING

Develop se directly main mat jao. Correct production path always: **develop -> staging -> main**.

6. Conflict aaye to kya karna hai

A conflict usually means Pawan aur Himani ne same lines ya same shared file ko change kiya.

```
git status
# Open each conflicted file and remove conflict markers
git add <resolved-files>
git rebase --continue
git status
git push origin develop
```

NOT SURE? STOP SAFELY

Agar correct resolution clear nahi hai, `git rebase --abort` chalao aur doosre developer ke saath file discuss karo.

Commands/actions to avoid

COMMAND	KYA KARTA HAI
<code>git push --force</code>	Shared history overwrite kar sakta hai. Ordino workflow mein use nahi karna.
Direct push to staging	Testing gate bypass hota hai. Staging only PR from develop.
Direct push to main	Production safety bypass hoti hai. Main only PR from staging.
<code>git reset --hard</code>	Uncommitted work permanently discard kar sakta hai. Team workflow mein avoid.
<code>git add .</code> without review	Accidentally secrets ya unrelated files stage ho sakti hain. Specific files add karo.
Commit .env or credentials	Security incident create kar sakta hai. Secrets kabhi Git mein nahi.

If push is rejected

```
git pull --rebase origin develop
# Resolve conflict if Git asks
git push origin develop
```

7. One-page daily cheat sheet

Morning / task start

```
git switch develop
git pull --rebase origin develop
git status
```

After coding

```
git status
git diff
git add <specific-files>
git diff --staged
git commit -m "feat(scope): clear message"
git pull --rebase origin develop
git push origin develop
```

Milestone to staging

```
gh pr create --base staging --head develop \
--title "release: promote develop to staging"
```

Approved staging to main

```
gh pr create --base main --head staging \
--title "release: promote staging to main"
```

FINAL MEMORY RULE

Pawan push to develop -> Himani pull develop -> Himani push develop -> PR develop to staging -> test -> PR staging to main.

Keep this PDF beside the repository while the workflow becomes a habit.